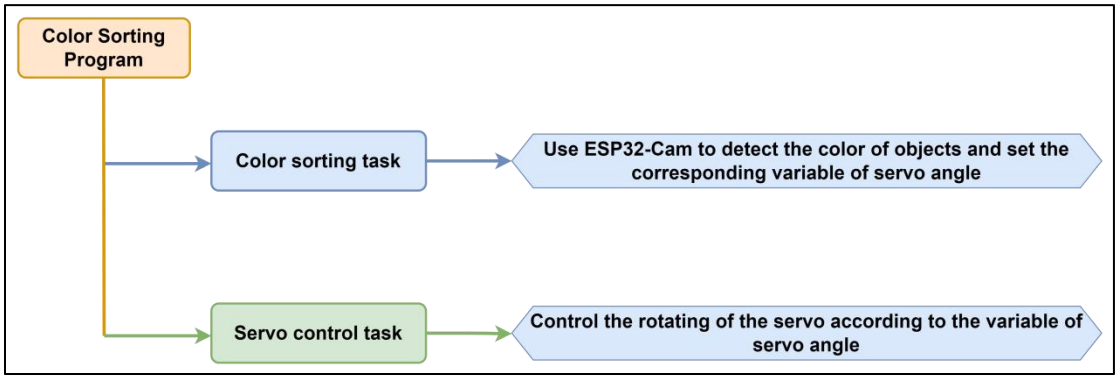
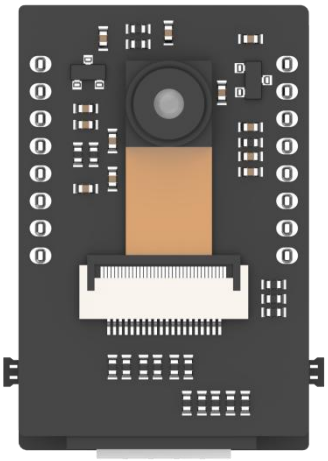
Color Sorting

In this section, miniArm will use the ESP32S3-Cam to recognize blue and green blocks, and then grasp and place them in their respective positions.

1. Program Flowchart



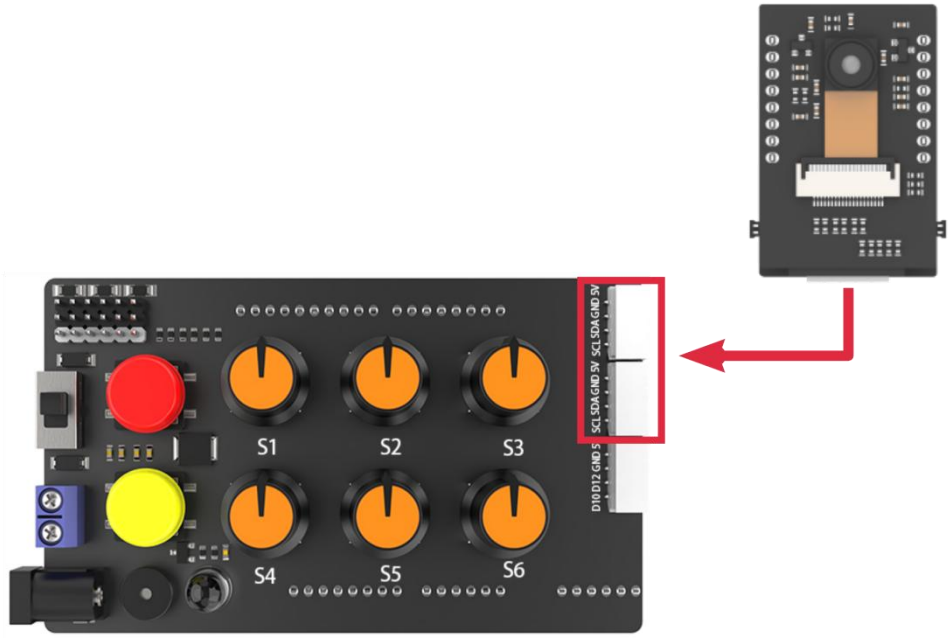
2. ESP32S3-Cam Development Board



ESP32S3-Cam development board integrates an ESP32 chip and a camera module. After being inserted into the expansion board, it uses the I2C communication interface to read color and face data through I2C communication.

2.1 Wiring

Align the pins of ESP32S3-Cam with the pin headers on the camera expansion board, and connect it to an I2C interface on the miniArm expansion board with a 4PIN wire.



3. Program Download

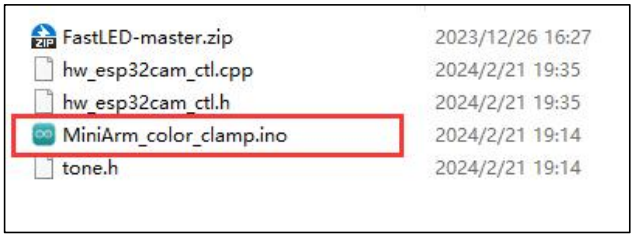
Note

◆ Remove the Bluetooth module before downloading the program. Or the program will fail to download because of the serial port conflict.

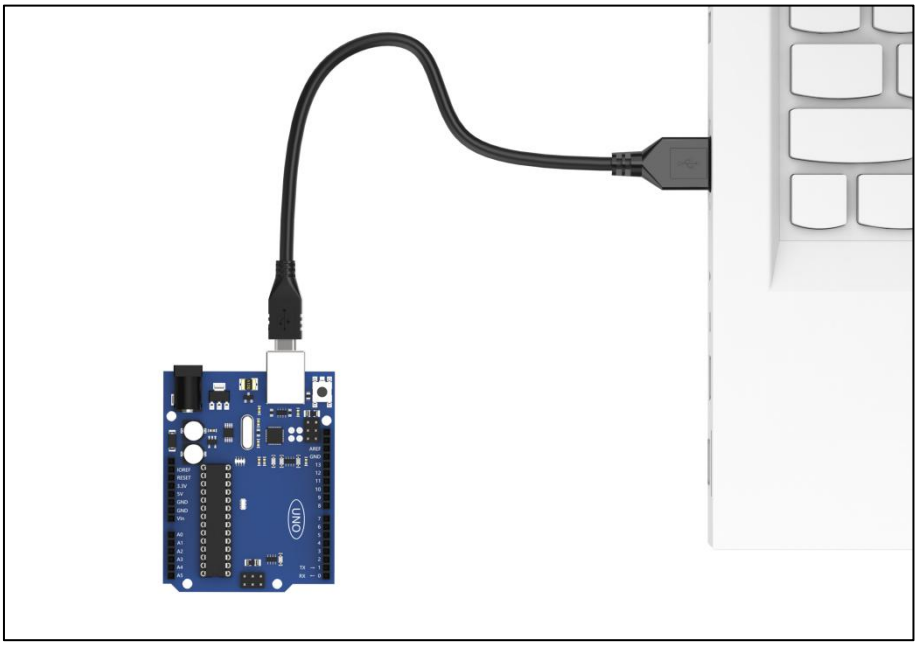
◆ Please switch the battery box to the “OFF” when connecting the Type-B download cable. This action prevents the download cable from accidentally touching the power pins of the expansion board, which may cause a short circuit.

3.1 Arduino UNO Program Download

1) Locate and open the “MiniArm_color_clamp\MiniArm_color_clamp.ino” program file in the same directory as this section.



2) Connect Arduino to the computer with the UNO cable (Type-B).



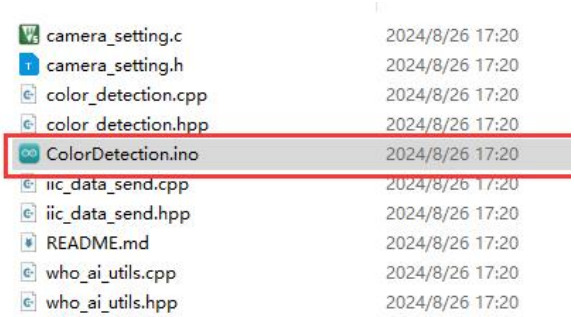
3) Click “Select Development Board”, and the software will automatically detect the current Arduino serial port. Next, click to connect.

4) Click to download the program into Arduino. Then just wait for it to complete.

3.2 ESP32S3-Cam Program Download

1)Connect the ESP32S3-Cam to the computer with a Type-C data cable.

2)Locate and open the “ColorDetection\ColorDetection.ino” program file in the same directory as this section.



3)Select the “ESP32S3 Dev Module” development board.

4)Click the “Tool” in the menu bar, and select the corresponding configuration for the ESP32S3 development board.

5)Click to download the program to the ESP32S3-Cam, and then wait for the download to complete.

4. Program Outcome

1) After being powered on, the robotic arm enters the initial position of grasping, and the RGB light on the expansion board lights up blue.

2) When a blue block is identified, the RGB lights up blue. The robotic arm grasps the block and places it to the left.

3) When a green block is identified, the RGB lights up green. The robotic arm grasps the block and places it to the right.

5. Brief Program Analysis

5.1 Import Library File

1) Import the RGB control library, servo control library, ESP32Cam communication library, and buzzer tone library files required for this program.

```
#include <FastLED.h> //import LED library
#include <Servo.h> //import servo library
#include "hw_esp32cam_ctl.h" //import ESP32Cam communication library
#include "tone.h" //tone library
```

2) Define tone and color constants.

```
const static uint16_t DOC5[] = { TONE_C5 };
const static uint16_t DOC6[] = { TONE_C6 };

#define COLOR_1   1
#define COLOR_2   2
```

5.2 Define Pin and Create Object

1) Firstly, define the Arduino pins used for hardware connections, mainly consisting of five servo pins, one buzzer pin, and one RGB light pin.

```
const static uint8_t servoPins[5] = { 7, 6, 5, 4, 3 };
const static uint8_t buzzerPin = 11;
const static uint8_t rgbPin = 13;
```

2) Then, define the control objects of RGB lights, ESP32Cam communication, action groups, and servos. The variables of the servo control are also created. The “limit_angle” sets angle limits for each joint of the robotic arm. The “servo_angles” array is used to store the actual angles of each servo, ranging from 0 to 180.

```
//RGB light control object
static CRGB rgbs[1];
//ESP32Cam communication object
HW_ESP32Cam hw_cam;
//action group control object
HW_ACTION_CTL action_ctl;
//servo control object
Servo servos[5];

/* The limit of each joint’s angle */
const uint8_t limt_angles[5][2] = {{0,82},{0,180},{0,180},{0,180},{0,180}};
/* The actual angle value of the servo control */
static uint8_t servo_angles[5] = { 40,21,159,115,90 };
```

3) The construction function of the action group’s control object defines the variable “extended_func_angles”, which is the desired angle value of servo angles. The related functions used to control executing action groups are also defined.

```
class HW_ACTION_CTL{
  public:
  /* the using angles of secondary development example */
    uint8_t extended_func_angles[5] = { 41 ,12 ,174 ,68 ,84 };
    //Control execution action group
    void action_set(int num);
    int action_state_get(void);
    void action_task(void);
    void read_offset();
    int8_t* get_offset(void);

  private:
    //Action group control variables
    int action_num = 0;
    int8_t servo_offset[5];
    uint8_t eeprom_read_buf[16];
};
```

Control executing action group

Control variable of action group

Regarding the desired and actual values, there may be differences between them during the process of controlling the servo rotation due to various factors. You can check in the “servo_control” function called in the “loop” main function.

If you want the servo to gradually move towards the target position, you need to call “servo_angles[i] = servo_angles[i] * 0.9 + extended_func_ angles[i] * 0.1” in the control task function.

This allows the servo to move each time at 90% of the actual value and 10% of the expected value, so that the actual value will gradually approach the expected value. And when it is equal to the expected value, the robotic arm stops moving.

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + action_ctl.extended_func_angles[i] * 0.1;
      if(servo_angles[i] < action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }else if(servo_angles[i] < action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (action_ctl.extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }
    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

4) Next, the variable used for controlling the buzzer is defined. Please note that task functions execute different control tasks. The “servo_control” function controls the servo. The “play_tune()” function sets the buzzer’s sound. The “tune_task” function controls the buzzer. And the “espcam_task” function processes the corresponding communication of the ESP32S3-Cam.

```
static void servo_control(void); /* servo control */
void play_tune(uint16_t *p, uint32_t beat, uint16_t len); /* buzzer control connector */
void tune_task(void); /* buzzer control task */
void espcam_task(void); /* esp32cam communication task */
```

5.3 Initialization

1) The “setup ()” function mainly initializes associated hardware devices. Starting with the serial port, which sets the baud rate as “115200” and a read timeout as 500ms.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  // set the timeout for reading data from the serial port
  Serial.setTimeout(500);
```

2) Assign the servo IO port to facilitate the control through the pin.

```
  // Assign the servo IO port
  for (int i = 0; i < 5; ++i) {
    servos[i].attach(servoPins[i]);
  }
```

3) Initialize the communication connector of ESP32S3-Cam.

```
hw_cam.begin(); //initialize the ESP32Cam communication connector
```

4) Use the “FastLED” library to initialize the RGB lights on the expansion board, and connect them to the RGB pins. Then set the color to blue by “rgbs[0] = CRGB(0, 0, 100)” and use the “FastLED.show” function to display the set color.

```
//Initialize and control the RGB light
FastLED.addLeds<WS2812, rgbPin, GRB>(rgbs, 1);
rgbs[0] = CRGB(0, 0, 100);
FastLED.show();
```

5) Set the pin of the buzzer to output mode and stop activating the buzzer after a short sound.

```
pinMode(buzzerPin, OUTPUT);
tone(buzzerPin, 1000);
delay(100);
noTone(buzzerPin);
```

6) Output the “Start" string through a serial port to indicate that the sensor is ready to start transmitting data.

```
delay(2000);
Serial.println("start");
```

5.4 Call Subfunction in a Loop

After finishing the initialization, it enters the “loop” main function. First, call the “espcam_task” function which is used to detect color identification and perform grasp and placement. Then, call the “tune_task” function to execute buzzer task. Next, call the “servo_control” function to control servos. Finally, call the “action_task()” to execute the action group.

```
void loop() {
  // esp32cam communication task
  espcam_task();
  // buzzer sound task
  tune_task();
  // servo control
  servo_control();
  // action group running task
  action_ctl.action_task();
}
```

5.5 ESP32S3-Cam Communication Task

1) In definition of the “espcam_task” function, the “last_tick” records the timestamp of the last task execution; the “step” tracks the current stage of task execution; the “act_num” records action group numbers to be called; the “delay_count” sets counting unit for the demonstration; and “color” stores detection result from the camera.

```
void espcam_task(void)
{
  static uint32_t last_tick = 0;
  static uint8_t step = 0;
  static uint8_t act_num = 0;
  static uint32_t delay_count = 0;
  int color = 0;
```

2) The “last_tick” variable and “millis()” are used for delay operation. Specifically, use the “millis()” function to obtain the running time of the program minus the “last_tick” variable. If the difference is less than 100, the function will return. If the difference is greater than or equal to 100, it means the task has been delayed by 100ms. Then the current time is assigned to the “last_tick” variable for the next delay.

```
// time intervals 100ms
if (millis() - last_tick < 100) {
  return;
}
last_tick = millis();
```

3) Perform corresponding tasks based on the detected color, such as controlling RGB LED, buzzer, and the angle of servo. Here is the analysis of this “switch” code snippet:

```
switch(step)
{
  case 0:
    color = hw_cam.colorDetect(); //obtain color
    if(color == COLOR_1) //if the color 1 is recognized
    {
      rgbs[0].r = 0;
      rgbs[0].g = 250;
      rgbs[0].b = 0;
```

Case 0: use “hw_cam.colorDetect()” to detect color. If color 1 is detected, RGB lights up red and the buzzer sounds to enter the next state. If blue is detected, same as this way.

```
  case 0:
    color = hw_cam.colorDetect(); //obtain color
    if(color == COLOR_1) //if the color 1 is recognized
    {
      rgbs[0].r = 0;
      rgbs[0].g = 250;
      rgbs[0].b = 0;
      FastLED.show();
      play_tune(DOC6, 300u, 1u);
      // to run action group 1 for color sorting
      act_num = 1;
      step++;
```

Turn off the RGB light if no target color is detected.

```
}else{ //If no target color is detected
  rgbs[0].r = 0;
  rgbs[0].g = 0;
  rgbs[0].b = 0;
  FastLED.show();
}
break;
```

Case 1: wait for some time before recording the running action group numbers, and proceeding to the next state.

```
case 1: //wait for 1s to place the block well
  delay_count++;
  if(delay_count > 10)
  {
    delay_count = 0;
    // run action group
    action_ctl.action_set(act_num);
    act_num = 0;
    step++;
  }
  break;
```

Case 2: wait for the action status to reset. If the action group running is finished, set the “step” to 0 to restart detecting the color.

```
case 2: //wait for the action status to reset
  if(action_ctl.action_state_get() == 0)
  {
    step = 0;
  }
  break;
default:
  step = 0;
  break;
```

5.6 Action Group Running Task

```
void action_task(void)
{
```

1) Initially, the variables for action group running are created: “last_tick” is used for a short delay; “step” is the current stage of action group running; “num” temporarily stores several actions in action group running; and “delay_count” is used for delay to wait for the ending of the action running.

```
static uint32_t last_tick = 0;
static uint8_t step = 0;
static uint8_t num = 0 , delay_count = 0;
```

2) Switch is the actual part of called action group. The detailed analysis is as below:

Case 0: To run action group. Transmit the action in the action group to the desired angle of the servo “extended_func_angles” to process to the next stage. If “num=0”, it means the action running is complete. The action group variable will be reset and jump out of the loop.

```
switch(step)
{
  case 0: //action running
    if(action[action_num-1][num][0] != 0)
    {
      extended_func_angles[0] = action[action_num-1][num][1] + servo_offset[0];
      extended_func_angles[1] = action[action_num-1][num][2] + servo_offset[1];
      extended_func_angles[2] = action[action_num-1][num][3] + servo_offset[2];
      extended_func_angles[3] = action[action_num-1][num][4] + servo_offset[3];
      extended_func_angles[4] = action[action_num-1][num][5] + servo_offset[4];

      step = 1;

    }else{ //If the running is complete
      num = 0;
      // Clean up the variable of action group
      action_num = 0;

    }
    break;
```

Case 1: use the “delay_count” variable to delay and wait for the action running. Then proceed “num+1” to case 0 to start executing the next action.

```
case 1: //wait for the action to run
  delay_count++;
  if(delay_count > 2)
  {
    num++;
    delay_count = 0;
    step = 0;
  }
  break;
```

5.7 Servo Control

If you want the servo to close to the target position, call 5 lines of codes

“servo_angles[i] = servo_angles[i] * 0.85 + extended_func_angles[i] * 0.15” in the control task function.

This allows the servo to move each time by 85% of the actual value and 15% of the desired value, so that the actual value gets progressively closer to the desired value. Once they are equal, the robotic arm stops moving.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();

  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + extended_func_angles[i] * 0.1;
      if(servo_angles[i] < extended_func_angles[i])
        servo_angles[i] = extended_func_angles[i];
    }else if(servo_angles[i] < extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > extended_func_angles[i])
        servo_angles[i] = extended_func_angles[i];
    }

    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

5.8 Buzzer Control

Control the buzzer to play a melody according to the specified rhythm and tone array.

```
void tune_task(void) {
  static uint32_t l_tune_beat = 0;
  static uint32_t last_tick = 0;
  // if it is not timed and it rings the same number as last time
  if (millis() - last_tick < l_tune_beat && tune_beat == l_tune_beat) {
    return;
  }
  l_tune_beat = tune_beat;
  last_tick = millis();
  if (tune_num > 0) {
    tune_num -= 1;
    tone(buzzerPin, *tune++);
  } else {
    noTone(buzzerPin);
    tune_beat = 10;
    l_tune_beat = 10;
  }
}
```

1) Declare variables. The “l_tune_beat” variable is used for storing the interval between the last played tones (in milliseconds); the “last_tick” variable is used for storing timestamp of the last function execution. Control the buzzer to play a melody according to specified rhythm and tone.

```
static uint32_t l_tune_beat = 0;
static uint32_t last_tick = 0;
```

2) Determine whether the tone needs to be played. Using the “millis()” function to get current timestamp and to compare with the “last_tick”. If the interval between current time and last played time is less than “l_tune_beat”, and the value of “tune_beat” does not change (i.e. “tune_beat” is equal to “l_tune_beat”), the function returns directly without executing any operation. This is to prevent the same tone from being played repeatedly in the same interval.

```
if (millis() - last_tick < l_tune_beat && tune_beat == l_tune_beat) {
  return;
}
```

3) Update variables. Assign the value of “tune_beat” to “l_tune_beat”, and update the “last_tick” as the current timestamp.

```
l_tune_beat = tune_beat;
last_tick = millis();
```

4) If “tune_num” is larger than 0, it means that there are still tones that need to be played. Then use the “tone()” function to play current tone in buzzer pin.

```
if (tune_num > 0) {
  tune_num -= 1;
  tone(buzzerPin, *tune++);
```

5) If “tune_num” is less than 0, it means all tones have been played. Use the “noTone()” function to stop the buzzer from sounding, and reset “tune_beat”

and “l_tune_beat” to 10 (that is, the buzzer task runs at an interval of 10ms) in preparation for the next playback of the melody.

```
  } else {
    noTone(buzzerPin);
    tune_beat = 10;
    l_tune_beat = 10;
  }
}
```

5.9 Buzzer Sound

Specify the tone array, the interval between the tones, and the number of tones to be played.

```
void play_tune(uint16_t *p, uint32_t beat, uint16_t len) {
  tune = p;
  tune_beat = beat;
  tune_num = len;
}
```

“tune = p” sets variable “tune” (a pointer to an array of tones) to the value of the passed parameter p. In this way, “tune” points to an array of tones provided by the user.

“tune_beat = beat” sets variable “tune_beat” (it means the interval between tones to be played) to the value of the passed parameter “beat”, which determines the tempo of the tone.

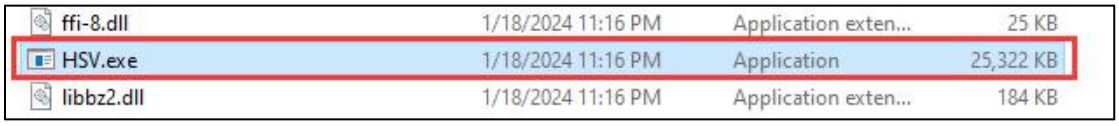
“tune_num = len” sets variable “tune_num” (it means the number of tones to be played) to the value of the passed parameter “len”, which determines the length of the playlist.

6. Function Extension

How to modify the recognized color of ESP32S3-Cam? You can refer to the following steps:

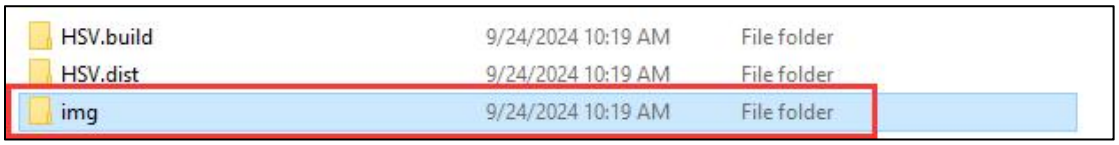
1) Locate and open the “8.4.2 ESP32S3-Cam Color Recognition

program\HSV\HSV.dist\HSV.exe" program file in the same path as this lesson.



ffi-8.dll	1/18/2024 11:16 PM	Application exten...	25 KB
HSV.exe	1/18/2024 11:16 PM	Application	25,322 KB
libbz2.dll	1/18/2024 11:16 PM	Application exten...	184 KB

2) Select the image file to be imported. The imported image needs to be stored in the "img" folder.

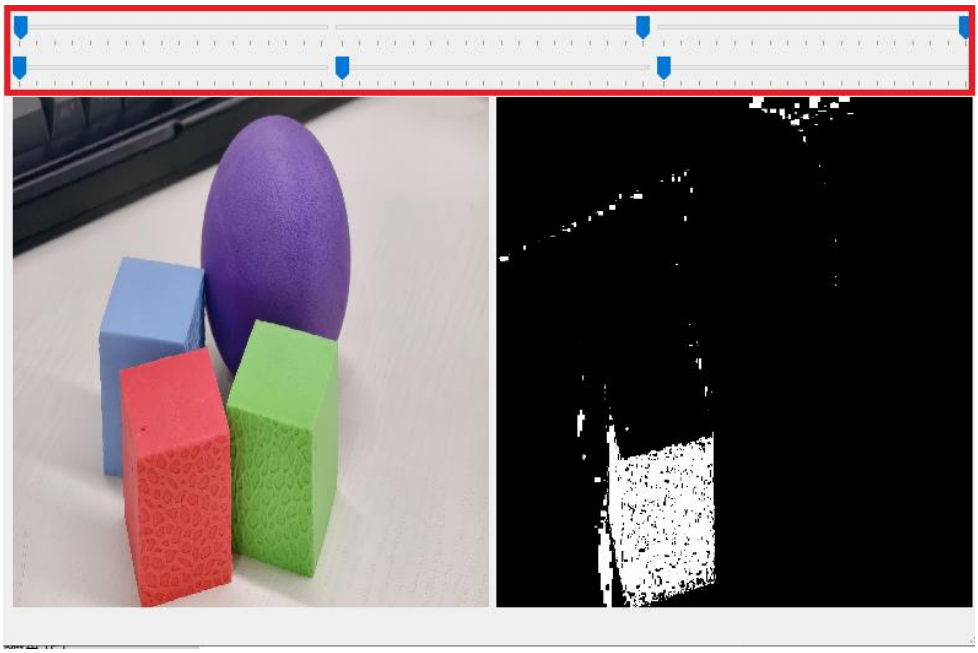


HSV.build	9/24/2024 10:19 AM	File folder
HSV.dist	9/24/2024 10:19 AM	File folder
img	9/24/2024 10:19 AM	File folder

3) Drag the slider to segment the image with HSV threshold, and adjust it to the appropriate HSV threshold range. You can refer to the following color range table.

Note: image stretching due to resolution does not affect threshold debugging.

	Black	Gtay	White	Red		Orange	Yellow	Green	Cyan	Blue	Purple
Hmin	0	0	0	0	156	11	26	35	78	100	125
Hmax	180	180	180	10	180	25	34	77	99	124	155
Smin	0	0	0	43		43	43	43	43	43	43
Smax	255	43	30	255		255	255	255	255	255	255
Vmin	0	46	221	46		46	46	46	46	46	46
Vmax	46	220	255	255		255	255	255	255	255	255



4) Save the HSV threshold. Locate and open the "8.4.2 ESP32S3-Cam Color Recognition Program\ColorDetection\color_detection.cpp" file in the same path as this lesson. Modify the color data to the saved HSV array. Next, refer to

“3.2 ESP32S3-Cam Program Download” to flash the modified program into ESP32S3-Cam.

```
vector<color_info_t> std_color_info = {
    {{151, 15, 70, 255, 90, 255}, 64, "red"},
    {{23, 34, 70, 255, 90, 255}, 64, "yellow"},
    {{45, 75, 70, 255, 90, 255}, 64, "green"},
    {{97, 117, 70, 255, 90, 255}, 64, "blue"},
    {{130, 155, 70, 255, 90, 255}, 64, "purple"}
};
```

Note: The elements of the array are separated with commas.

5) Once the flashing is completed, the ESP32S3-Cam can recognize the object with other colors.

7. FAQ

Q:Since the code was upload, the color can not be recognized.

A:Please check that you have connected the 4-pin cable to the I2C interface and turn the knob on the expansion board to the initial position.

Q:Why is the color recognized by the camera inaccurate or even incorrect?

A:Please try to minimize the complexity of the background. You can use a single color or a simple environment as the background.